

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA1220	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Moniga V	Reg. No.:	192521184
Section / Batch:	2025 - 2029	Date:	26/08/2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system continuously receives large blocks of data from a high-speed I/O device. Write pseudocode to design a DMA-based data transfer algorithm that transfers data directly from the I/O device to main memory with minimal CPU involvement. The algorithm should initialize the DMA controller, configure source and destination addresses, specify the transfer size, initiate the transfer, and handle completion and error interrupts.
2. An embedded system is connected to multiple I/O devices, each capable of generating interrupts with different priority levels. Develop pseudocode for an interrupt handling mechanism that identifies and prioritizes vectored interrupts, determines the interrupting device, and dispatches the appropriate interrupt service routine. The algorithm should ensure that high-priority interrupts are serviced efficiently while maintaining proper handling of lower-priority requests.
3. A computer system communicates with different I/O devices having varying latency, bandwidth, and datatransfer requirements. Write pseudocode for a dynamic I/O selection mechanism that chooses between memory-mapped I/O and I/O-mapped I/O based on the performance requirements of each device. The algorithm should evaluate latency and bandwidth requirements and select the mapping technique that provides efficient communication.
4. A computer system contains multiple devices that simultaneously request access to high-speed communication interfaces such as PCIe, USB, and Thunderbolt. Design pseudocode for a bus arbitration algorithm that manages competing access requests and allocates the appropriate interface to each

device. The algorithm should consider device priority, bandwidth requirements, interface availability, and fairness to ensure efficient utilization of communication resources.

5. A real-time embedded system communicates with both low-speed devices, such as simple sensors, and high-speed devices, such as data acquisition units and storage devices. Write pseudocode for a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupt-driven communication for high-speed devices. The algorithm should identify the device type, select the appropriate I/O technique, initiate data transfer, and handle completion and error conditions while minimizing CPU involvement.

Code of Conduct

I Moniga V(192521184) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Moniga V

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge

Question 1: DMA-Based Data Transfer Algorithm

1. Problem Understanding

A high-speed I/O device continuously produces large data blocks that must reach main memory with minimal CPU involvement. The algorithm must: initialize the DMA controller, configure source (device) and destination (memory) addresses, specify the transfer size, start the transfer, and correctly handle both a successful-completion interrupt and an error interrupt (with retry logic).

2. Pseudocode

```
BEGIN DMA_Transfer(source_device, dest_memory_addr, transfer_size)
  // Step 1: Check controller availability
  IF DMA_Controller.status == BUSY THEN
    RETURN ERROR "DMA controller busy"
  END IF

  // Step 2: Initialize and configure DMA registers
  DMA_Controller.SOURCE_ADDR = source_device.data_register
  DMA_Controller.DEST_ADDR   = dest_memory_addr
  DMA_Controller.TRANSFER_SIZE = transfer_size
  DMA_Controller.MODE         = BLOCK_TRANSFER
  DMA_Controller.INTERRUPT_ENABLE = TRUE

  // Step 3: Initiate transfer and free the CPU
  DMA_Controller.CONTROL_REG = START_TRANSFER
  CPU.continue_other_tasks()  // CPU does NOT wait / poll
```

```

    RETURN "Transfer initiated"
END

// Interrupt Service Routine triggered by the DMA controller
ISR DMA_Completion_Handler(source_device, dest_memory_addr, transfer_size)
    IF DMA_Controller.STATUS_REG == TRANSFER_ERROR THEN
        LOG_ERROR("DMA transfer failed", DMA_Controller.ERROR_CODE)
        DMA_Controller.CONTROL_REG = ABORT
        Retry_Count = Retry_Count + 1

        IF Retry_Count < MAX_RETRIES THEN
            CALL DMA_Transfer(source_device, dest_memory_addr, transfer_size)
        ELSE
            NOTIFY_SYSTEM("DMA transfer permanently failed")
            Retry_Count = 0
        END IF

    ELSE IF DMA_Controller.STATUS_REG == TRANSFER_COMPLETE THEN
        Buffer_Ready_Flag = TRUE
        Process_Received_Data(dest_memory_addr, transfer_size)
        DMA_Controller.CONTROL_REG = CLEAR_INTERRUPT
        Retry_Count = 0
    END IF

    RETURN FROM INTERRUPT
END

```

3. Control Structures Used

- IF / ELSE IF: to branch on controller availability and on completion vs. error status.
- Function/Module decomposition: DMA_Transfer() (setup) is separated from the DMA_Completion_Handler() ISR (event response), reflecting the real hardware separation between initiation and interrupt-driven completion.
- Recursive retry call: on error, DMA_Transfer() is re-invoked up to MAX_RETRIES, bounded by a counter to avoid infinite retries.

4. Completeness / Edge Cases Handled

- Busy-controller check prevents overlapping/conflicting transfers.
- Both completion and error interrupts are explicitly handled (not just the success path).
- A bounded retry mechanism (MAX_RETRIES) avoids an infinite error loop and escalates to system notification on persistent failure.
- The CPU never busy-waits — it is released immediately after initiating the transfer, satisfying the minimal-CPU-involvement requirement.

Question 2: Prioritized Vectored Interrupt Handling

1. Problem Understanding

Multiple I/O devices can each raise an interrupt with a distinct priority level. The mechanism must identify which device interrupted (via a vector), dispatch directly to the correct ISR, ensure higher-priority interrupts are serviced promptly (even pre-empting a running lower-priority ISR), and still guarantee lower-priority requests are eventually handled — not dropped.

2. Pseudocode

```
BEGIN System_Init()
    Interrupt_Vector_Table[0..N-1] = preconfigured_ISR_addresses
    Priority_Level[0..N-1]          = assigned_device_priorities    // 0 = highest
    Current_Running_Priority        = LOWEST_PRIORITY
    Pending_Interrupt_Queue         = EMPTY_QUEUE
END

ISR Master_Interrupt_Handler()
    device_id          = Interrupt_Controller.GET_VECTOR()    // hardware-supplied
                        vector
    requested_priority  = Priority_Level[device_id]

    IF requested_priority < Current_Running_Priority THEN
        // Higher priority than what is currently executing -> pre-empt (nested
        // interrupt)
        PUSH_CONTEXT(Current_Running_Priority)
        Previous_Priority = Current_Running_Priority
        Current_Running_Priority = requested_priority
        ENABLE_HIGHER_PRIORITY_INTERRUPTS()

        ISR_Address = Interrupt_Vector_Table[device_id]    // vectored dispatch
        CALL ISR_Address()

        Current_Running_Priority = Previous_Priority
        POP_CONTEXT()
    ELSE
        // Equal / lower priority than current work -> queue for later
        ENQUEUE(Pending_Interrupt_Queue, device_id, requested_priority)
    END IF

    RETURN FROM INTERRUPT
END

FUNCTION Service_Pending_Queue()
    WHILE Pending_Interrupt_Queue NOT EMPTY DO
        next_device = DEQUEUE_HIGHEST_PRIORITY(Pending_Interrupt_Queue)
        Current_Running_Priority = Priority_Level[next_device]
```

```
    ISR_Address = Interrupt_Vector_Table[next_device]
    CALL ISR_Address()

    Current_Running_Priority = LOWEST_PRIORITY
END WHILE
END
```

3. Control Structures Used

- IF / ELSE: decides whether to pre-empt immediately or enqueue for later service.
- WHILE loop: drains the pending-interrupt queue in priority order once the CPU is free.
- Array indexing (Interrupt_Vector_Table[device_id]): implements true vectored dispatch — no sequential polling of devices.
- Function modules: System_Init(), Master_Interrupt_Handler(), and Service_Pending_Queue() separate configuration, real-time dispatch, and deferred servicing.

4. Completeness / Edge Cases Handled

- Nested interrupts are supported through explicit context push/pop around pre-emption.
- Lower-priority interrupts are never lost — they are queued and later dequeued by priority order, so no request is starved indefinitely.
- Vector-table lookup replaces polling, minimizing dispatch latency for the highest-priority devices.
- Current_Running_Priority is always restored after an ISR completes, keeping the priority state consistent across nested calls.

Question 3: Dynamic I/O Mapping Selection (Memory-Mapped vs I/O-Mapped)

1. Problem Understanding

Different devices have different latency and bandwidth needs. The algorithm must evaluate each device's requirements and automatically decide whether Memory-Mapped I/O or I/O-Mapped I/O gives more efficient communication, then configure the address decoder accordingly.

2. Pseudocode

```
FUNCTION Select_IO_Mapping(device)
    latency_req      = device.max_acceptable_latency
    bandwidth_req    = device.required_bandwidth
    access_frequency = device.access_frequency

    IF latency_req < LATENCY_THRESHOLD AND access_frequency == HIGH THEN
        // Time-critical, frequently accessed -> avoid extra instruction overhead
        mapping = MEMORY_MAPPED_IO

    ELSE IF bandwidth_req > BANDWIDTH_THRESHOLD THEN
        // High-bandwidth devices benefit from DMA + flexible addressing modes
        mapping = MEMORY_MAPPED_IO

    ELSE IF device.requires_isolated_address_space == TRUE THEN
        // Legacy / low-frequency device needing a separate I/O space
        mapping = IO_MAPPED_IO

    ELSE
        // Default: modern systems favor memory-mapped I/O
        mapping = MEMORY_MAPPED_IO
    END IF

    CONFIGURE_ADDRESS_DECODER(device, mapping)
    RETURN mapping
END

FUNCTION Evaluate_All_Devices(device_list)
    FOR EACH device IN device_list DO
        chosen_mapping = Select_IO_Mapping(device)
        device.mapping_type = chosen_mapping
        LOG("Device " + device.id + " assigned " + chosen_mapping)
    END FOR
END
```

3. Control Structures Used

- IF / ELSE IF / ELSE chain: evaluates latency, then bandwidth, then legacy-isolation needs, with a sensible default fallback.

- FOR EACH loop: applies the selection function across an arbitrary list of devices, making the algorithm scale to any number of devices.
- Function decomposition: the per-device decision logic (Select_IO_Mapping) is separated from the batch driver (Evaluate_All_Devices) for reusability.

4. Completeness / Edge Cases Handled

- A default ELSE branch guarantees every device receives a mapping decision — no device is left unclassified.
- Both latency-sensitive and bandwidth-sensitive cases are checked independently, since a device could qualify under either criterion.
- Legacy devices requiring an isolated address space are still correctly routed to I/O-mapped I/O even if their latency/bandwidth needs are modest.
- The address decoder is reconfigured for every decision, so the hardware state always matches the algorithm's choice.

Question 4: Bus Arbitration Algorithm (PCIe / USB / Thunderbolt)

1. Problem Understanding

Multiple devices simultaneously request access to shared high-speed interfaces (PCIe, USB, Thunderbolt). The arbitration algorithm must allocate interfaces based on device priority, bandwidth needs, and current interface availability, while remaining fair — no request should be starved indefinitely.

2. Pseudocode

```
STRUCTURE Request
    device_id, interface_type, priority, bandwidth_needed, wait_time
END STRUCTURE

FUNCTION Add_Request(device_id, interface_type, priority, bandwidth)
    new_req = CREATE Request(device_id, interface_type, priority, bandwidth,
    wait_time = 0)
    ENQUEUE(Request_Queue, new_req)
END

FUNCTION Bus_Arbitrator(Request_Queue)
    WHILE SYSTEM_RUNNING DO
        IF Request_Queue NOT EMPTY THEN

            // Step 1: Age every waiting request to prevent starvation
            FOR EACH req IN Request_Queue DO
                req.wait_time = req.wait_time + 1
                req.effective_priority = req.priority - (req.wait_time /
                AGING_FACTOR)
            END FOR

            // Step 2: Pick the request with the best (lowest) effective priority
            selected_req = SELECT_MIN(Request_Queue, by = effective_priority)
            target_interface = selected_req.interface_type

            // Step 3: Check interface availability and remaining bandwidth
            IF Interface_Available[target_interface] == TRUE AND
            Interface_Bandwidth_Remaining[target_interface] >=
            selected_req.bandwidth_needed THEN

                GRANT_BUS_ACCESS(selected_req.device_id, target_interface)
                Interface_Bandwidth_Remaining[target_interface] -=
                selected_req.bandwidth_needed
                REMOVE(Request_Queue, selected_req)

                PERFORM_TRANSFER(selected_req)

                // Release bandwidth once transfer completes
```

```

        Interface_Bandwidth_Remaining[target_interface] +=
        selected_req.bandwidth_needed
    ELSE
        // Interface busy or insufficient bandwidth right now - retry next
        cycle
        CONTINUE
    END IF
END IF
END WHILE
END

```

3. Control Structures Used

- WHILE SYSTEM_RUNNING: continuously services requests as long as the system is active.
- FOR EACH loop: applies aging to every queued request each arbitration cycle.
- IF / ELSE: checks interface availability and bandwidth before granting access; otherwise the request stays queued for the next cycle.
- SELECT_MIN by effective priority: combines static priority with an aging term, implemented as a comparison/selection operation over the queue.

4. Completeness / Edge Cases Handled

- The aging term ($\text{wait_time} / \text{AGING_FACTOR}$) guarantees that a long-waiting low-priority request eventually gets the lowest effective-priority value and is serviced — preventing starvation.
- Bandwidth is explicitly reserved on grant and released on completion, preventing over-allocation of a shared interface.
- If the chosen interface is busy or lacks bandwidth, the algorithm defers rather than blocking, allowing other ready requests to still be considered in future cycles.
- The structure is interface-agnostic (`interface_type` is a parameter), so it uniformly supports PCIe, USB, and Thunderbolt requests in the same queue.

Question 5: Hybrid Polling–DMA I/O Communication Mechanism

1. Problem Understanding

The system has both low-speed devices (simple sensors) and high-speed devices (data-acquisition units, storage). The algorithm must classify each device by type, use lightweight polling for low-speed devices, and use DMA with interrupt-driven completion for high-speed devices — minimizing CPU involvement overall while correctly handling completion and error conditions.

2. Pseudocode

```
FUNCTION Hybrid_IO_Manager(device_list)
  FOR EACH device IN device_list DO
    IF device.type == LOW_SPEED THEN
      CALL Polling_Handler(device)
    ELSE IF device.type == HIGH_SPEED THEN
      CALL DMA_Interrupt_Handler(device)
    ELSE
      LOG_ERROR("Unknown device type", device.id)
    END IF
  END FOR
END

FUNCTION Polling_Handler(device)
  LOOP EVERY poll_interval DO
    status = READ(device.status_register)    // memory-mapped access

    IF status == DATA_READY THEN
      data = READ(device.data_register)
      Process_Sensor_Data(device.id, data)
    ELSE IF status == ERROR THEN
      LOG_ERROR(device.id, status)
    END IF
    // status == NOT_READY -> simply loop again next interval
  END LOOP
END

FUNCTION DMA_Interrupt_Handler(device)
  IF DMA_Controller.status == BUSY THEN
    ENQUEUE(High_Speed_Wait_Queue, device)
    RETURN
  END IF

  CONFIGURE_DMA(device.source_addr, device.buffer_addr, device.transfer_size)
  DMA_Controller.CONTROL_REG = START_TRANSFER
  // CPU is now free -- no blocking wait
  RETURN
END
```

```

ISR DMA_Completion_ISR(device)
  IF device.status == TRANSFER_ERROR THEN
    LOG_ERROR(device.id, "DMA transfer failed")
    Retry_Count[device.id] = Retry_Count[device.id] + 1

    IF Retry_Count[device.id] < MAX_RETRIES THEN
      CALL DMA_Interrupt_Handler(device)
    ELSE
      NOTIFY_SYSTEM(device.id, "Permanent DMA failure")
    END IF

  ELSE IF device.status == TRANSFER_COMPLETE THEN
    Process_Bulk_Data(device.buffer_addr, device.transfer_size)
    RESET_DMA(device)
    Retry_Count[device.id] = 0

    IF High_Speed_Wait_Queue NOT EMPTY THEN
      next_device = DEQUEUE(High_Speed_Wait_Queue)
      CALL DMA_Interrupt_Handler(next_device)
    END IF
  END IF

  RETURN FROM INTERRUPT
END

```

3. Control Structures Used

- FOR EACH loop: classifies and dispatches every device in the system exactly once.
- IF / ELSE IF / ELSE: routes each device to the correct handler by type, and separately branches DMA completion vs. error.
- LOOP EVERY poll_interval: implements periodic (non-blocking-in-spirit) polling for low-speed devices without needing hardware interrupts.
- Queue (High_Speed_Wait_Queue): defers a high-speed request if the single DMA controller is already busy, then services it via the same handler once the controller frees up.

4. Completeness / Edge Cases Handled

- Unknown/unclassified device types are logged rather than silently ignored.
- A busy DMA controller does not drop a high-speed request — it is queued and automatically resumed from within the completion ISR.
- Both transfer-error and transfer-complete outcomes are explicitly handled for high-speed devices, with a bounded retry count as in Question 1.
- Low-speed devices use simple polling (appropriate given their low data rate and low CPU cost), while high-speed devices use DMA + interrupts, so CPU involvement is minimized exactly where it matters most (the high-bandwidth path).

Summary of Algorithms

Q.No	Algorithm	Core Technique(s)
1	DMA-based data transfer	DMA setup/config + completion & error ISR with bounded retries
2	Prioritized interrupt handling	Vectored dispatch + priority pre-emption + pending queue
3	Dynamic I/O mapping selection	Rule-based decision on latency / bandwidth / isolation needs
4	Bus arbitration	Priority + aging + bandwidth/availability check across PCIe/USB/Thunderbolt
5	Hybrid polling–DMA I/O	Polling for low-speed devices, DMA+interrupt for high-speed devices